

Simulation task

1 .Simulation of edge computing delay under local processing, cloud processing, latency, bandwidth, and task size

```
clc;
```

```
clear;
```

```
close all;
```

```
% Task size in MB
```

```
TaskSize = 1:10:100;
```

```
% System parameters
```

```
CPU_Edge = 2e9;    % Edge CPU frequency (cycles/sec)
```

```
CPU_Cloud = 10e9;  % Cloud CPU frequency (cycles/sec)
```

```
CyclesPerBit = 1000; % CPU cycles required per bit
```

```
Bandwidth = 10e6;   % Bandwidth = 10 Mbps
```

```
Latency = 50e-3;    % Network latency = 50 ms
```

```
% Convert task size from MB to bits
```

```
TaskBits = TaskSize * 8e6;
```

```
% Processing delay
```

```
EdgeProcessingDelay = (TaskBits * CyclesPerBit) / CPU_Edge;
```

```
CloudProcessingDelay = (TaskBits * CyclesPerBit) / CPU_Cloud;
```

```
% Transmission delay
```

```
TransmissionDelay = TaskBits / Bandwidth;
```

```
% Total delay for local processing
```

```
LocalDelay = EdgeProcessingDelay;
```

```
% Total delay for cloud processing
```

```
CloudDelay = Latency + TransmissionDelay + CloudProcessingDelay;
```

```
% Display results
```

```
disp('Task Size (MB)   Local Delay (sec)   Cloud Delay (sec)');
```

```
for i = 1:length(TaskSize)
```

```
    fprintf('%10d %20.4f %20.4f\n', ...
```

```
        TaskSize(i), LocalDelay(i), CloudDelay(i));
```

```
end
```

```
% Plot delay comparison
```

```
figure;
```

```
plot(TaskSize, LocalDelay, '-o', 'LineWidth', 2);
```

```
hold on;
```

```
plot(TaskSize, CloudDelay, '-s', 'LineWidth', 2);
```

```
grid on;
```

```
xlabel('Task Size (MB)');
```

```
ylabel('Total Delay (seconds)');
```

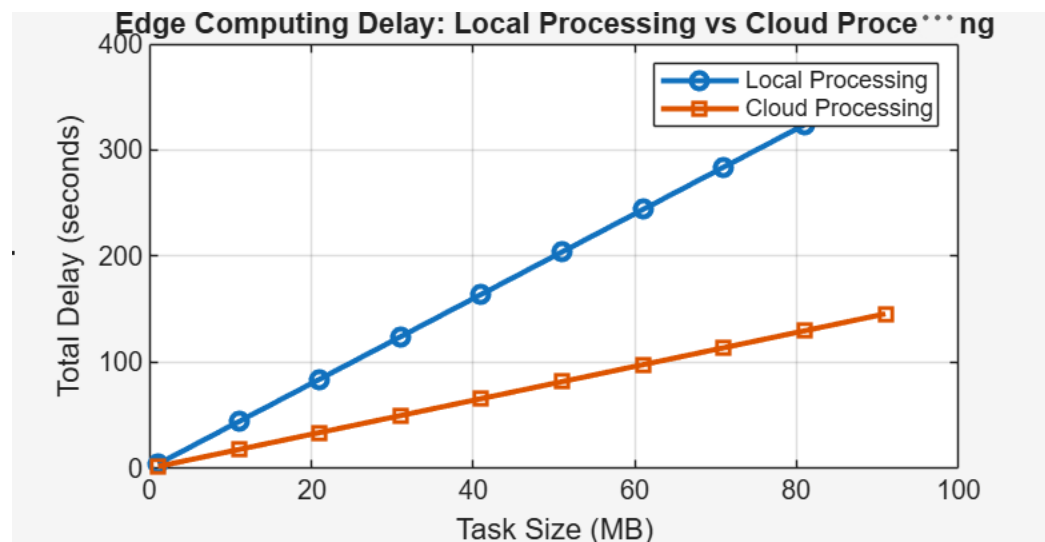
```
title('Edge Computing Delay: Local Processing vs Cloud Processing');
```

```
legend('Local Processing', 'Cloud Processing');
```

output

Task Size (MB)	Local Delay (sec)	Cloud Delay (sec)
----------------	-------------------	-------------------

1	4.0000	1.6500
11	44.0000	17.6500
21	84.0000	33.6500
31	124.0000	49.6500
41	164.0000	65.6500
51	204.0000	81.6500
61	244.0000	97.6500
71	284.0000	113.6500
81	324.0000	129.6500
91	364.0000	145.6500



2 .Simulation of carrier aggregation performance in LTE-Advanced under single carrier, dual carrier, triple carrier, varying bandwidth, and SNR

```
clc;
```

```
clear;
```

```
close all;
```

```
% SNR range in dB
```

```
SNR = 0:5:30;
```

```
% Bandwidth of each carrier in MHz
```

```
BW = 20;
```

```
% Number of carriers
```

```
N1 = 1;    % Single Carrier
```

```
N2 = 2;    % Dual Carrier
```

```
N3 = 3;    % Triple Carrier
```

```
% Calculate Spectral Efficiency using Shannon Capacity
```

```
SE = log2(1 + 10.^(SNR/10));
```

```
% Throughput calculation (Mbps)
```

```
Throughput_Single = N1 * BW * SE;
```

```
Throughput_Dual   = N2 * BW * SE;
```

```
Throughput_Triple = N3 * BW * SE;
```

```
% Display results
```

```
disp('SNR(dB)   Single Carrier(Mbps)   Dual Carrier(Mbps)   Triple Carrier(Mbps)');
```

```

for i = 1:length(SNR)
    fprintf('%5d %20.2f %20.2f %20.2f\n', ...
        SNR(i), ...
        Throughput_Single(i), ...
        Throughput_Dual(i), ...
        Throughput_Triple(i));
end

% Plot Carrier Aggregation Performance
figure;

plot(SNR, Throughput_Single, '-o', 'LineWidth', 2);
hold on;
plot(SNR, Throughput_Dual, '-s', 'LineWidth', 2);
plot(SNR, Throughput_Triple, '-^', 'LineWidth', 2);

grid on;

xlabel('SNR (dB)');
ylabel('Throughput (Mbps)');

title('LTE-Advanced Carrier Aggregation Performance');

legend('Single Carrier', 'Dual Carrier', 'Triple Carrier', ...
    'Location', 'northwest');

```

SNR(dB)	Single Carrier(Mbps)	Dual Carrier(Mbps)	Triple Carrier(Mbps)
0	20.00	40.00	60.00
5	41.15	82.29	123.44
10	69.19	138.38	207.57
15	100.56	201.11	301.67
20	133.16	266.33	399.49
25	166.19	332.38	498.56
30	199.34	398.69	598.03

